Bölüm 3: Fonksiyonlar

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week3_part2_Functions
7 Kasım 2026

İçindekiler

Bölüm 3 · Fonksiyonlar 3

Bölümün amacı 3

Konu anlatımı 3

Fonksiyon nedir? (Slayt 1–6) 3

Tanımlama ve çağırma (Slayt 7–13) 3

Yerel değişkenler ve kapsam (Slayt 14–15) 4

Argüman ve parametre (Slayt 16–25) 5

Global değişkenler ve sabitler (Slayt 26–28) 5

Standart kütüphane ve random modülü (Slayt 29–38) 6

return ile değer döndürmek (Slayt 39–45) 6

math modülü (Slayt 46–48) 7

Kendi modüllerin ve __name__ (Slayt 49–56) 8

Sık yapılan hatalar 8

Kısa tekrar soruları 9

Cevaplar 9

Bölüm 3 · Fonksiyonlar

Bölümün amacı

Bu bölüm programı küçük, yeniden kullanılabilir parçalara bölmeyi öğretir. Sonunda şunları yapabilmelisin:

- Fonksiyon tanımlamak (`def`), çağırmak ve bir `main` fonksiyonuyla programı düzenlemek
- Yerel değişken, kapsam, argüman ve parametre kavramlarını ayırt etmek
- Pozisyonel ve anahtar kelimeli argüman kullanmak; global değişkenlerin risklerini bilmek
- `return` ile değer, Boolean, birden çok değer ve `None` döndürmek
- `random` ve `math` modüllerini, kendi modüllerini ve `__name__ == '__main__'` kalıbını kullanmak

Konu anlatımı

Fonksiyon nedir? (Slayt 1–6)

Fonksiyon, belirli bir görevi yerine getiren ifadeler grubudur. Büyük bir işi küçük görevlere bölüp her birini bir fonksiyona vermeye **böl ve yönet** denir; böyle yazılmış programa **modüler program** denir. Faydaları: daha basit kod, kodun yeniden kullanımı, daha kolay test ve ekip çalışması.

İki tür fonksiyon vardır:

- **Void fonksiyon:** ifadelerini çalıştırır ve biter.
- **Değer döndüren fonksiyon:** çağıran ifadeye bir değer geri gönderir (`input`, `int`, `float` gibi).

Tanımlama ve çağırma (Slayt 7–13)

Fonksiyon adı değişken adı kurallarına uyar ve genellikle bir fiil içerir. Tanım iki parçadır: **başlık** (`def ad():`) ve **girintili blok** (gövde). Fonksiyon **çağrıldığında** yorumlayıcı gövdeye atlar, ifadeleri çalıştırır ve çağrıldığı yere geri döner.

Programın ana mantığını bir `main` fonksiyonuna yazıp diğer fonksiyonları ondan çağırmak yaygın bir düzendir. Henüz yazılmamış fonksiyonlar `pass` ile iskelet olarak bırakılabilir.

PYTHON

```python
def main():
    print('Başla')
    selamla()
    hazirla()
    print('Bitir')

def selamla():
    print('Merhaba!')

def hazirla():
    pass   # daha sonra yazılacak

main()
```

ÇIKTI

```
Başla
Merhaba!
Bitir
```

Bloktaki satırlar aynı sayıda boşlukla girintilenmeli; sekme (Tab) ve boşluk **karıştırılmamalıdır**.

Yerel değişkenler ve kapsam (Slayt 14–15)

Bir fonksiyonun içinde değer atanan değişken **yereldir**: yalnızca o fonksiyondaki ifadeler erişebilir. Bir değişkene erişilebilen program bölümüne **kapsam** (scope) denir. Farklı fonksiyonlarda aynı isimli yerel değişkenler birbirini etkilemez.

PYTHON

```python
def texas():
    birds = 5000
    print('texas:', birds)

def california():
    birds = 8000
    print('california:', birds)

texas()
california()
```

ÇIKTI

```
texas: 5000
california: 8000
```

Argüman ve parametre (Slayt 16–25)

Argüman, fonksiyona gönderilen veridir; **parametre** onu alan değişkendir. Birden çok argüman **sırayla** (pozisyonel) eşleşir. **Anahtar kelimeli argümanla** (`parametre=deger`) sıra önemsiz hale gelir; karışık kullanımda pozisyonel argümanlar önce gelir.

Parametreye fonksiyon içinde yeni değer atamak, çağıranın değişkenini **değiştirmez** (değerle aktarım, pass by value).

PYTHON

```python
def show_sum(num1, num2):
    print(num1 + num2)

def show_interest(principal, rate, periods):
    print(principal * rate * periods)

def change_me(arg):
    arg = 0
    print('içeride:', arg)

show_sum(12, 45)
show_interest(rate=0.01, periods=10, principal=10000.0)
value = 99
change_me(value)
print('dışarıda:', value)
```

ÇIKTI

```
57
1000.0
içeride: 0
dışarıda: 99
```

Global değişkenler ve sabitler (Slayt 26–28)

Tüm fonksiyonların dışında oluşturulan değişken **globaldir**; her yerden okunabilir. Bir fonksiyon global değişkene **atama** yapacaksa içinde `global ad` bildirimi gerekir.

Global değişkenler hata ayıklamayı ve programı anlamayı zorlaştırır, fonksiyonları onlara bağımlı kılar; bu yüzden kaçınılmalıdır. Değişmeyen **global sabitler** ise uygundur.

PYTHON

```python
my_value = 10

def show_value():
    print(my_value)
```

PYTHON (DEVAM)

```python
def change():
    global my_value
    my_value = 50

show_value()
change()
show_value()
```

ÇIKTI

```
10
50
```

Standart kütüphane ve random modülü (Slayt 29–38)

Standart kütüphane, Python ile gelen hazır fonksiyonlardır; bunları **kara kutu** gibi kullanırız. Bazıları yerleşiktir (`print`, `range`), diğerleri **modüllerdedir** ve `import modul` ile yüklenip `modul.fonksiyon()` nokta gösterimiyle çağrılır.

`random` modülü: `randint(a, b)` (iki uç dahil), `randrange(...)` (range gibi), `random()` (0.0–1.0 arası float), `uniform(a, b)` (aralıkta float). Bu sayılar **sözde rastgeledir**: aynı **tohum** (`seed`) aynı seriyi üretir.

PYTHON

```python
import random
random.seed(10)
print(random.randint(1, 100), random.randrange(0, 10, 2))
print(round(random.random(), 4), round(random.uniform(1, 5), 4))
random.seed(10)
print(random.randint(1, 100))   # aynı tohum → aynı sayı
```

ÇIKTI

```
74 0
0.4289 3.3124
74
```

return ile değer döndürmek (Slayt 39–45)

`return ifade`, ifadenin değerini çağıran yere gönderir. Dönen değer bir değişkene atanabilir ya da başka bir fonksiyona verilebilir.

- **Boolean fonksiyon** True/False döndürür; koşulları ve girdi doğrulamayı sadeleştirir.
- **Birden çok değer:** `return a, b`; çağıran tarafta her değer için bir değişken gerekir.

- `None` "değer yok" demektir; ör. sıfıra bölmede hata işareti olarak döndürülebilir.

PYTHON
```python
def toplam(num1, num2):
    return num1 + num2

def is_even(n):
    return n % 2 == 0

def divide(num1, num2):
    if num2 == 0:
        return None
    return num1 / num2

def get_name():
    return 'Ada', 'Lovelace'

print(toplam(12, 45), is_even(4), divide(10, 0), divide(10, 4))
first, last = get_name()
print(last, first)
```

ÇIKTI
```
57 True None 2.5
Lovelace Ada
```

math modülü (Slayt 46–48)

`import math` ile karekök (`sqrt`), yukarı/aşağı yuvarlama (`ceil`, `floor`), üstel ve logaritma (`exp`, `log`, `log10`), trigonometri (radyan), `hypot` ve `math.pi`, `math.e` sabitleri kullanılabilir.

PYTHON
```python
import math
print(math.sqrt(16), math.ceil(4.2), math.floor(4.8))
print(math.hypot(3, 4), round(math.degrees(math.pi), 1))
radius = 2
print(round(math.pi * radius ** 2, 2))
```

ÇIKTI
```
4.0 5 4
5.0 180.0
12.57
```

Kendi modüllerin ve `__name__` (Slayt 49–56)

Modülerleştirme, ilgili fonksiyonları `.py` dosyalarında gruplamaktır; kodu anlamayı, test etmeyi ve yeniden kullanmayı kolaylaştırır. Bir modül fonksiyon **tanımları** içerir, çağrıları değil. **Menü güdümlü** programlar kullanıcıya işlemler listesi gösterip seçime göre çalışır.

Bir dosya yüklendiğinde `__name__` değişkeni oluşur: dosya doğrudan çalıştırıldıysa `'__main__'`, import edildiyse modülün adıdır. Böylece `main()` yalnızca dosya ana program olduğunda çalışır:

PYTHON
```python
def main():
    print('Ana program çalıştı, __name__ =', __name__)

if __name__ == '__main__':
    main()
```

ÇIKTI
```
Ana program çalıştı, __name__ = __main__
```

Sık yapılan hatalar

- **Fonksiyonu tanımlanmadan önce çağırmak** → `NameError`.

PYTHON
```python
selamla()
def selamla():
    print('Merhaba')
```

ÇIKTI
```
NameError: name 'selamla' is not defined
```

- **Argüman sayısını tutturamamak** → `TypeError`.

PYTHON
```python
def topla(a, b):
    return a + b
print(topla(5))
```

ÇIKTI
```
TypeError: topla() missing 1 required positional argument: 'b'
```

- **Yerel değişkene fonksiyon dışından erişmeye çalışmak** → `NameError`.
- `global` **bildirimi olmadan global değişkene** `+=` **yapmak** → `UnboundLocalError`.

- `return` **yazmayı unutmak:** fonksiyon `None` döndürür; `print(f())` "None" yazar.
- `import math` **yapmadan** `math.pi` **kullanmak** → `NameError`.

Kısa tekrar soruları

1. Void fonksiyon ile değer döndüren fonksiyonun farkı nedir?
2. `def f(a, b, c)` için `f(1, c=3, b=2)` geçerli midir? Peki `f(c=3, 1, 2)`?
3. Fonksiyon içinde parametreye yeni değer atamak çağıranın değişkenini değiştirir mi?
4. Global değişkenlerden neden kaçınılır?
5. `random.seed(5)` iki kez çağrılıp her seferinde `randint(1, 10)` istenirse sonuçlar aynı mı olur?
6. `if __name__ == '__main__':` kalıbı ne işe yarar?

Cevaplar

1. Void fonksiyon yalnızca ifadelerini çalıştırıp biter; değer döndüren fonksiyon çağıran ifadeye `return` ile bir değer gönderir.
2. İlki geçerlidir (pozisyonel önce, sonra anahtar kelimeli). İkincisi `SyntaxError` verir: pozisyonel argüman anahtar kelimeli argümandan sonra gelemez.
3. Hayır; değerle aktarım tek yönlüdür.
4. Hata ayıklamayı ve programı anlamayı zorlaştırır, fonksiyonları başka programlara taşımayı zorlaştırır.
5. Evet; aynı tohum aynı sözde rastgele seriyi üretir.
6. Dosya import edildiğinde `main()`'in otomatik çalışmasını önler; yalnızca doğrudan çalıştırıldığında çalışır.